

Teensy 4.1을 활용한 이동형 능동 자세 제어 시스템

Real-Time Ball Position Stabilization Using PID Control

터치패널을 이용한 공 위치 검출 및 능동 자세 제어 시스템 구현

26年 6月

한경국립대학교

전자공학과

2021740040 우성훈

2021740056 장경호

초록	4
서론	4
1.1 작품 제작 동기	4
1.2 기존 현황 및 문제점 제기	5
1.3 작품의 목표	5
본론	6
2.1 주요 부품	6
- 2.1.1 Teensy 4.1	6
- 2.1.2 MG996R Servo Motor	7
- 2.1.3 4-Wire Resistive Touch Panel	8
- 2.1.4 MPU6050	9
- 2.1.5 L298N 모터 드라이버	9
- 2.1.6 블루투스 모듈(HC-06)	10
2.2 설계	11
- 2.2.1 시스템 구성도	11
- 2.2.2 PID 제어 이론	12
- 2.2.3 역기구학(Inverse Kinematics)	13
- 2.2.4 MPU6050을 이용한 자세 보정	15
- 2.2.5 전체 회로도	16
2.3 구현방법	17
- 2.3.1 터치패널을 이용한 공 위치 검출	17
- 2.3.2 PID 제어 알고리즘 구현	19
- 2.3.3 역기구학 계산 구현	20
- 2.3.4 서보모터 제어	21
- 2.3.5 MPU6050 수평 보정	22
- 2.3.6 Ball & Plate 제어 알고리즘 구현	22
- 2.3.7 이동형 플랫폼 적용	23
2.4 주요기능 실험	24
- 2.4.1 터치패널 위치 검출 실험	24

– 2.4.2 역기구학 및 서보모터 동작 실험	26
– 2.4.3 PID 게인 튜닝 실험	26
– 2.4.4 MPU6050 자세 보정 실험	27
– 2.4.5 Ball Balancing 실험	29
– 2.4.6 외란 복원 실험	30
– 2.4.7 이동형 플랫폼 주행 실험	31
– 2.4.8 최종 통합 테스트	31

결론 33

3.1 결과 및 고찰 33

참고 문헌 34

부록	35
A. main.cpp 통합코드	35
B. Inverse kinematics .h 소스코드	42
C. Inverse kinematics .cpp 소스코드	43

초록

제약·화학·반도체 등 정밀 산업에서는 운반 과정에서 발생하는 적재물의 미세한 흔들림이 품질 저하와 유출 사고로 직결되며, 특히 액체와 같이 자유 표면(free surface)이 출렁이는 적재물은 운반체의 가속·감속·회전 시 발생하는 슬로싱(sloshing)으로 인해 제어가 매우 까다롭다. 이러한 적재물은 단순히 고정하거나 운반 속도를 늦추는 방식만으로는 안정적으로 운반하기 어려우며, 이동 중에도 플랫폼의 자세를 실시간으로 감지하고 능동적으로 보정하는 정밀 제어 기술이 요구된다.

본 작품에서는 이러한 능동 자세 제어 원리를 검증하기 위하여, 점성 감쇠가 없어 제어가 가장 까다로운 자유 표면 물체로서 '평판 위의 공'을 대상으로 선정하였다. Stewart Platform을 단순화한 3RPS 병렬 매니플레이터(3자유도) 구조를 기반으로 공의 위치를 실시간으로 검출·제어하는 Ball Balancing System을 설계하였으며, 4선식 저항막 터치패널로 공의 위치를 측정하고 PID 제어와 역기구학 알고리즘을 적용하여 플랫폼의 자세를 제어함으로써 공을 중심 위치에 유지하였다. 또한 MPU6050 관성센서를 이용하여 이동 중 발생하는 외란을 보정함으로써, 정지 환경뿐 아니라 주행 환경에서도 공의 균형을 유지할 수 있는 시스템을 구현하였다.

서론

1.1 작품 제작 동기

제약·화학·반도체 등 정밀 산업에서는 운반 과정에서 발생하는 적재물의 미세한 흔들림조차 품질 저하나 유출 사고로 직결된다. 특히 액체처럼 형상을 고정할 수 없고 자유 표면(free surface)이 출렁이는 적재물은, 운반체가 가속·감속·회전할 때마다 내부에서 슬로싱(sloshing)이 발생하여 제어가 매우 까다롭다.

이러한 적재물은 용기에 단단히 고정하더라도 유체 자체의 출렁임을 막을 수 없으며, 단순히 운반 속도를 늦추는 방식만으로는 생산성과 안정성을 동시에 만족시키기 어렵다는 한계가 있다. 따라서 운반체가 이동하는 중에도 적재물의 상태와 플랫폼의 자세를 실시간으로 감지하고, 외란을 상쇄하도록 플랫폼의 자세를 능동적으로 보정하는 정밀 제어 기술이 요구된다.

본 작품은 이러한 능동 자세 제어의 원리를 검증하기 위하여, 자유 표면 물체 중에서도 제어가 가장 까다로운 극단적 사례로 '평판 위의 공'을 선정하였다. 공은 내부 점성에 의한 감쇠가 거의 없고 미세한 기울기에도 즉시 굴러 이동하는 한계 안정(marginally stable) 시스템이므로, 이를 이동 환경에서 안정적으로 중심에 유지할 수 있다면 동일한 보정 원리를 슬로싱 억제를 비롯한 다양한 정밀 운반 문제로 확장할 수 있다고 보았다.

1.2 기존 현황 및 문제점 제기

운반 중 발생하는 슬로싱을 억제하기 위한 능동 제어 연구는 최근 활발히 진행되고 있다. 퍼지 로직 제어를 이용하여 정밀 유체 운반 장치의 슬로싱을 억제하거나, 심층 강화학습(DRL) 기반 피드백 제어로 개방형 용기의 액체 출렁임과 이동 시간을 동시에 최소화하는 연구 등이 보고되었다. 그러나 이러한 시스템은 대부분 특정 용기·유체 모델에 맞추어 설계되어 범용성이 떨어지고, 고가의 액추에이터와 복잡한 동역학 모델링을 필요로 하여 제어 원리 자체를 학습·검증하기에는 진입 장벽이 높다.

한편 Ball & Plate 시스템은 PID 제어, 상태 제어, 인공지능 제어 등 다양한 제어 알고리즘을 검증하기 위한 대표적인 불안정 제어 플랫폼으로 널리 활용되어 왔다. 그러나 기존 사례들을 조사한 결과, 대부분 고정된 받침대 위에서 공의 위치 제어만을 수행할 뿐 운반체의 자세 정보를 활용하지 않으며, 운반체가 실제로 이동하는 환경에서의 외란 보정까지 다루는 경우는 드물었다. 또한 고가의 부품이나 복잡한 구조로 제작되어 제작 비용이 높고 구현이 어렵다는 문제점도 있었다.

따라서 비교적 저렴한 비용으로 구현할 수 있으면서도, 단순히 정지 상태의 위치 제어에 그치지 않고 운반체의 이동에 따른 외란을 실시간으로 보정할 수 있는 능동 자세 제어 시스템의 필요성이 제기되었다.

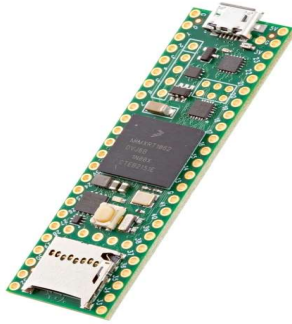
1.3 작품의 목표

본 작품은 Stewart Platform에서 파생된 3RPS 병렬 매니플레이터(3자유도) 구조를 기반으로 한 Ball & Plate 시스템을 이동형 플랫폼에 탑재하여, 운반체가 주행하는 중에도 공을 중심 위치에 안정적으로 유지하는 능동 자세 제어 시스템의 구현을 목표로 하였다. 터치패널로 공의 위치를 실시간 측정하고, PID 제어 알고리즘으로 목표 위치와의 오차를 계산하여 플랫폼의 목표 기울기를 산출한다. 이를 역기구학(Inverse Kinematics)을 통해 세 개의 서보모터 회전 각도로 변환하고, Teensy 4.1을 메인 제어기로 사용하여 실시간 제어가 가능하도록 시스템을 구성하였다.

특히 본 작품에서는 MPU6050 관성센서로 주행 중 발생하는 차체의 기울기와 진동을 측정하여 PID 출력에 반영함으로써, 정지 환경뿐 아니라 이동 환경에서도 외란을 보정하도록 설계하였다. 이를 통해 점성 감쇠가 없는 가장 가혹한 조건의 자유 표면 물체(공)를 이동 중에 제어할 수 있음을 보임으로써, 향후 정밀 운반 분야의 능동 자세 보정 기술로 확장될 수 있는 기초 플랫폼을 제시하는 것을 목표로 하였다.

본론

2.1 주요 부품



- 2.1.1 Teensy 4.1

Feature	Teensy 2.0	Teensy++ 2.0	Teensy LC	Teensy 3.2	Teensy 3.5	Teensy 3.6	Teensy 4.0	Teensy 4.1	Units
Processor Core	ATMEGA32U4 AVR	AT90USB1286 AVR	MKL26Z64VFT4 Cortex-M0+	MK20DX256VLH7 Cortex-M4	MK64FX512VMD12 Cortex-M4F	MK66FX1M0VMD18 Cortex-M4F	IMXRT1062DVL6 Cortex-M7	IMXRT1062DVL6 Cortex-M7	
FPU	-	-	-	-	32	32	32 & 64	32 & 64	bits
Rated Speed	16	16	48	72	120	180	600	600	MHz
Overclockable	-	-	-	96	-	240	912	912	MHz
Flash Memory	31.5	127	62	256	512	1024	1984	7936	kbytes
Bandwidth	32	32	96	192	192	411	66	66	Mbytes/sec
Cache	-	-	64	256	256	8192	65536	65536	Bytes
RAM	2.5	8	8	64	256	256	1024	1024	kbytes
EEPROM	1024	4096	128 (emu)	2048	4096	4096	1080 (emu)	4284 (emu)	bytes
Direct Memory Access	-	-	4	16	16	32	32	32	Channels
Digital I/O	25	46	27	34	58	58	40	55	Pins
Breadboard I/O	22	36	24	24	40+2	40+2	24	42	Pins
Voltage Output	5V	5V	3.3V / 5V	3.3V	3.3V	3.3V	3.3V	3.3V	Volts
Current Output	20mA	20mA	5mA / 20mA	10mA	10mA	10mA	10mA	10mA	milliAmps
Voltage Input	5V	5V	3.3V Only	5V Tolerant	5V Tolerant	3.3V Only	3.3V Only	3.3V Only	Volts
Interrupts	4	8	18	34	58	58	40	55	Pins
Analog Input	12	8	13	21	27	25	14	18	Pins
Converters	1	1	1	2	2	2	2	2	
Usable Resolution	10	10	12	13	13	13	10	10	Bits
Prog Gain Amp	1	1	-	2	-	-	-	-	
Touch Sensing	1	-	11	12	-	11	-	-	Pins
Comparators	1	1	1	3	3	4	4	4	
Analog Output	-	-	1	1	2	2	-	-	Pins
DAC Resolution	-	-	12	12	12	12	-	-	Bits
Timers	4 Total	4 Total	7 Total	12 Total	17 Total	19 Total	49 Total	49 Total	
PWM, 32 bit	-	-	-	-	-	-	3	3	
PWM, 16 bit	2	2	3	3	4	6	32	32	
PWM, 8-10 bit	2	2	-	-	-	-	-	-	
Total PWM Outputs	7	9	10	12	20	22	27	31	Pins
PDB Type	-	-	-	1	1	1	-	-	
CMT Type	-	-	-	1	1	1	-	-	
Quadrature Enc	-	-	-	-	-	-	4	4	
LPIMR Type	-	-	1	1	1	1	-	-	
PIT/Interval	-	-	2	4	4	4	4	4	
IEEE1588	-	-	-	-	4	4	4	4	
Systick	-	-	1	1	1	1	1	1	
RTC	-	-	0 **	1 **	1	1	1	1	
Communication									
USB	1	1	1	1	1	2	2	2	
Serial	1	1	3	3	6	6	7	8	
With FIFOs	-	-	-	2	2	2	7	8	
High Res Baud	-	-	-	3	6	5	-	-	
SPI	1	1	2	1	3	3	2	2	
With FIFOs	-	-	1	1	1	1	2	2	
I2C	1	1	2	2	3	4	3	3	
CAN Bus	-	-	-	1	1	2	3	3	
With CAN-FD	-	-	-	-	-	-	1	1	
Digital Audio In	-	-	1	2	2	2	5*	5*	stereo pins
Digital Audio Out	-	-	1	2	2	2	5*	5*	stereo pins
S/PDIF Input	-	-	-	-	-	-	1	1	
S/PDIF Output	-	-	-	0*	0*	0*	1	1	
MQS Output	-	-	-	-	-	-	1	1	
SD Card	-	-	-	-	1	1	1*	1	
Ethernet	-	-	-	-	1*	1*	-	1	uu

Teensy 4.1 사양 (자료: PJRC [PJRC Datasheets Page](#))

Teensy 4.1은 ARM Cortex-M7 기반의 고성능 마이크로컨트롤러로 최대 600MHz의 동작 속도를 지원한다. 또한 1024KB RAM과 8MB Flash Memory를 제공하며 다양한 PWM 출력과 입출력 핀을 지원한다. 본 작품에서는 터치패널로부터 공의 위치 정보를 수집하고 PID 제어 연산 및 역기구학 계산을 실시간으로 수행해야 하므로 높은 연산 성능이 요구된다. 따라서 빠른 처리 속도와 다양한 입출력 기능을 지원하는 Teensy 4.1을 메인 제어기로 선정하였다.

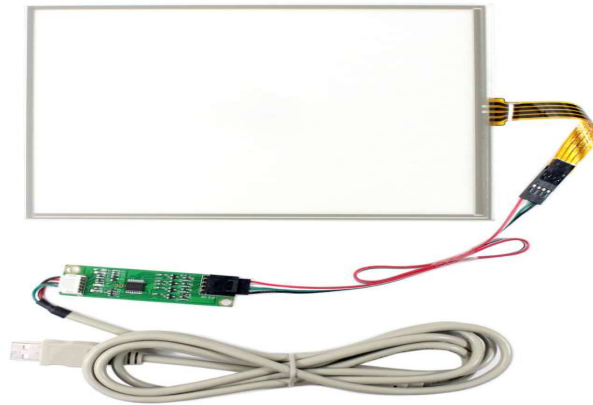
- 2.1.2 MG996R Servo Motor



MG996R 서보모터 스펙 (자료: <https://towerpro.com.tw/product/mg996r>)

MG996R은 금속 기어를 사용하는 고토크 서보모터로 높은 출력과 빠른 응답속도를 제공한다. 본 작품에서는 Stewart Platform의 자세를 제어하기 위해 3개의 MG996R 서보모터를 사용하였으며, 플랫폼의 기울기를 조절하여 공의 위치를 제어하는 역할을 수행한다.

- 2.1.3 4-Wire Resistive Touch Panel

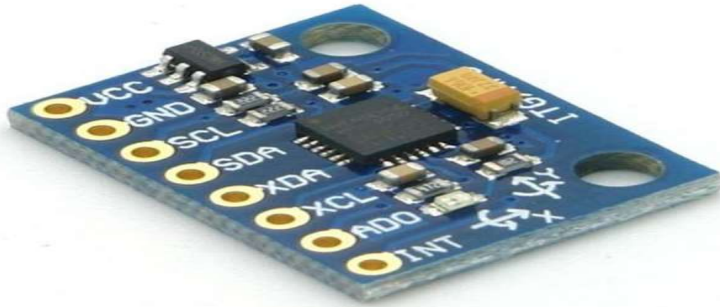


Ball & Plate 시스템에서 쇠구슬의 위치를 실시간으로 측정하기 위하여 4선식 저항막 방식 터치패널(4-Wire Resistive Touch Panel)을 사용하였다. 4선식 저항막 터치패널은 X+, X-, Y+, Y-의 네 개 전극으로 구성되며, 외부에서 압력이 가해질 경우 상·하부의 저항막이 접촉하여 위치에 따른 전압이 발생한다. 생성된 전압은 마이크로 컨트롤러의 ADC(Analog-to-Digital Converter)를 통해 디지털 값으로 변환되며, 이를 이용하여 X축과 Y축 좌표를 계산할 수 있다.

본 작품에서는 8.4인치 저항막 방식 터치패널을 플랫폼 상부에 설치하여 쇠구슬의 위치를 검출하였다. 터치패널의 X축 및 Y축 전압을 순차적으로 측정함으로써 쇠구슬의 현재 좌표를 실시간으로 획득할 수 있으며, 측정된 좌표 데이터는 PID 제어기의 입력값으로 사용된다. 이를 통해 플랫폼 위에서 이동하는 쇠구슬의 위치를 지속적으로 추적하고 목표 위치와의 오차를 계산하여 제어에 활용할 수 있다.

저항막 방식 터치패널은 정전식 터치패널과 달리 접촉 물체의 전기적 특성에 크게 영향을 받지 않기 때문에 손가락뿐만 아니라 금속 물체, 스타일러스 등 다양한 물체의 위치를 검출할 수 있다. 또한 구조가 비교적 단순하고 높은 위치 분해능을 제공하며, 실시간 좌표 측정이 가능하다는 장점을 가진다. 따라서 본 작품에서는 쇠구슬의 위치를 안정적으로 측정하기 위한 센서로 4선식 저항막 터치패널을 채택하였다.

2.1.4 MPU6050 관성센서

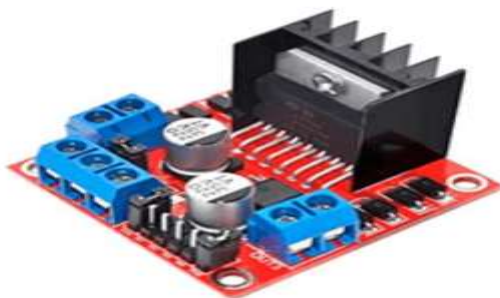


(자료: [MPU6050 Datasheet](#))

MPU6050은 3축 가속도계(Accelerometer)와 3축 자이로스코프(Gyroscope)가 하나의 칩에 통합된 6축 관성측정장치(IMU)이다. 가속도계는 물체의 기울기와 가속도를 측정하고, 자이로스코프는 회전 속도를 측정한다. 본 작품에서는 이동형 플랫폼 주행 시 발생하는 기울기와 진동을 측정하기 위하여 MPU6050을 사용하였다.

MPU6050은 I2C 통신을 이용하여 Teensy 4.1과 데이터를 주고받으며, 측정된 가속도 및 각속도 데이터를 상보필터(Complementary Filter)를 통해 처리하여 Roll과 Pitch 각도를 계산하였다. 계산된 기울기 값은 PID 제어에 반영되어 플랫폼의 수평 상태를 유지하도록 사용하였다.

2.1.5 L298N 모터 드라이버

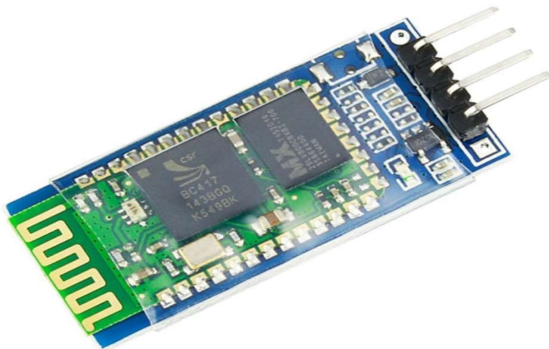


L298N 모터 드라이버 (자료:[STMicroelectronics L298 Datasheet](#))

L298N은 Dual H-Bridge 구조를 이용하여 DC 모터의 속도와 회전 방향을 제어할 수 있는 모터 드라이버 모듈이다. 마이크로컨트롤러의 출력 전류만으로는 모터를 직접 구동할 수 없기 때문에 본 작품에서는 L298N을 이용하여 이동형 플랫폼의 좌우 DC 모터를 제어하였다.

L298N은 PWM 신호를 이용하여 모터의 속도를 조절할 수 있으며, 입력 핀의 조합에 따라 정회전과 역회전이 가능하다. 본 작품에서는 블루투스 모듈을 통해 수신한 주행 명령을 기반으로 L298N을 제어하여 전진, 후진, 좌회전 및 우회전을 구현하였다.

2.1.6 블루투스 모듈 (HC-06)



HC-06 블루투스 모듈은 Bluetooth V2.0+EDR 기반의 무선 시리얼 통신 모듈로 UART 통신을 이용하여 마이크로컨트롤러와 스마트폰 간의 데이터를 송수신할 수 있다. 본 작품에서는 스마트폰 애플리케이션에서 전송된 전진(F), 후진(B), 좌회전(L), 우회전(R), 정지(S) 명령을 Teensy 4.1로 전달하기 위해 사용하였다. HC-06은 Slave 모드만 지원하므로 스마트폰과 연결하여 원격 제어를 수행하는 용도로 적합하다.

2.2 설계

2.2.1 시스템 구성도

플로우차트는 본 작품의 전체 시스템 동작 흐름도를 나타낸다. 시스템은 크게 위치 검출부, 자세 보정부, 제어 연산부, 구동부 및 이동 플랫폼으로 구성된다.

시스템이 시작되면 Teensy 4.1 제어기는 터치패널, MPU6050 관성센서, 서보모터 및 블루투스 통신 모듈을 초기화한다. 이후 MPU6050 센서의 초기 자세 값을 측정하여 Roll 및 Pitch 오프셋을 보정하고 기준 자세를 설정한다.

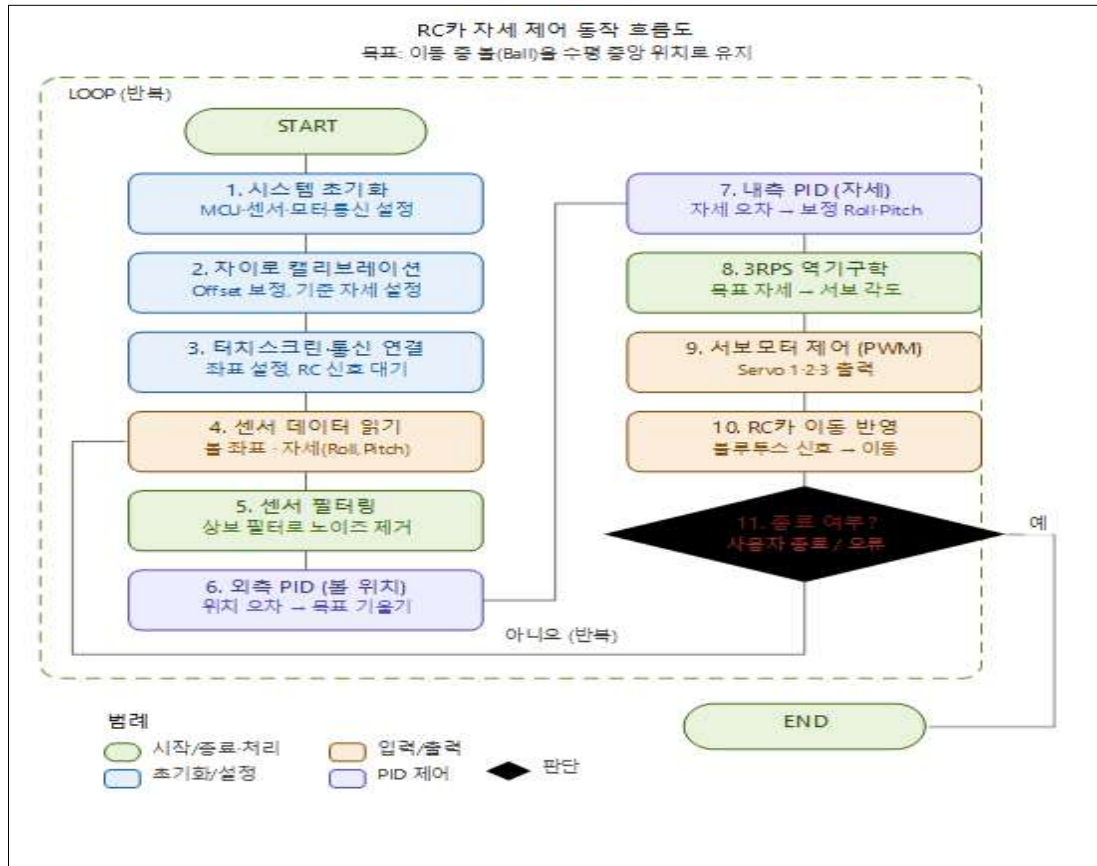
초기화가 완료되면 터치패널을 이용하여 쇠구슬의 현재 위치를 실시간으로 측정한다. 측정된 좌표값은 저역통과필터(Low Pass Filter)를 통해 노이즈를 제거한 후 제어기의 입력값으로 사용된다. 제어기는 목표 위치와 현재 위치의 오차를 계산하고, X축 및 Y축 방향에 대해 각각 PD 제어 연산을 수행한다.

동시에 MPU6050 관성센서는 이동 플랫폼의 기울기와 진동 정보를 측정한다. 측정된 Roll 및 Pitch 값은 PD 제어 출력에 반영되어 이동 중 발생하는 외란을 보상하도록 설계하였다. 이를 통해 차량의 가속, 감속 및 회전 과정에서 발생하는 자세 변화를 최소화할 수 있다.

PD 제어를 통해 계산된 목표 기울기 값은 역기구학(Inverse Kinematics) 알고리즘의 입력으로 사용된다. 역기구학 알고리즘은 목표 기울기를 각 서보모터의 회전 각도로 변환하며, 계산된 각도는 PWM 신호를 통해 3개의 서보모터에 전달된다. 서보모터는 플랫폼의 자세를 조절하여 쇠구슬이 목표 위치로 이동하도록 유도한다.

또한 블루투스 통신을 이용하여 이동 플랫폼의 주행 방향을 제어할 수 있도록 구성하였다. 사용자는 스마트폰을 통해 전진, 후진, 좌회전 및 우회전 명령을 전송할 수 있으며, 이동 중에도 Ball Balancing System이 지속적으로 동작하여 쇠구슬의 위치를 안정적으로 유지한다.

이와 같은 과정을 반복 수행함으로써 쇠구슬의 위치를 실시간으로 제어하며, 외란이 발생하더라도 목표 위치를 유지할 수 있는 능동 자세 제어 시스템을 구현하였다.



2.2.2 PID 제어 알고리즘 구현

PID(Proportional Integral Derivative) 제어는 목표값(Set Point)과 현재값(Current Value)의 오차(Error)를 이용하여 시스템을 제어하는 대표적인 폐루프(Closed Loop) 제어 방식이다. PID 제어는 비례 제어(P), 적분 제어(I), 미분 제어(D)로 구성되며 다양한 산업용 제어 시스템에서 사용된다.

본 작품에서는 플랫폼 위의 쇠구슬을 목표 위치에 유지하기 위하여 PID 제어 알고리즘을 적용하였다. 터치패널을 통해 쇠구슬의 현재 위치를 측정한 후 목표 위치와의 오차를 계산하고, 계산된 오차를 이용하여 플랫폼의 기울기를 결정하였다.

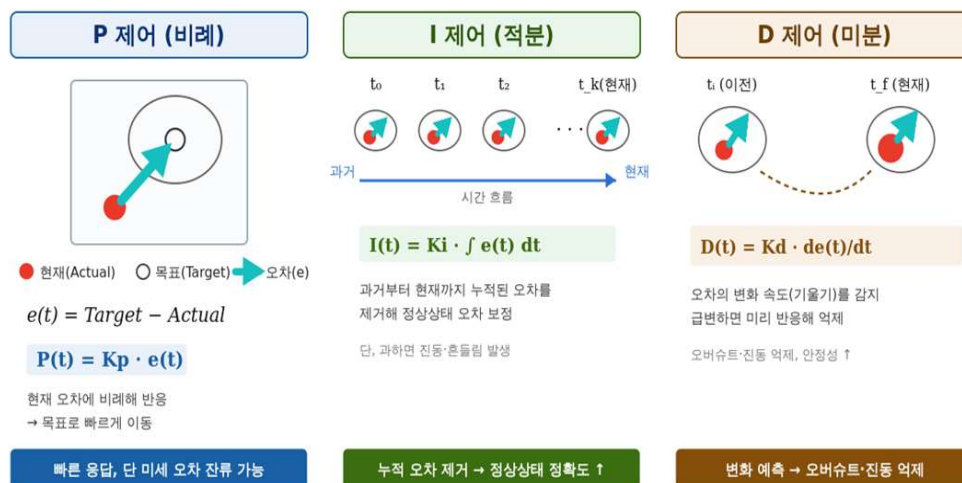
PID 제어의 동작 원리를 나타낸다. 비례 제어(P)는 현재 오차에 비례하여 제어량을 생성한다. 따라서 쇠구슬이 목표 위치에서 멀어질수록 더 큰 보정값을 생성하게 된다. 비례 제어는 빠른 응답성을 제공하지만 계인 값이 과도하게 증가할 경우 진동이나 오버슈트가 발생할 수 있다.

적분 제어(I)는 과거부터 현재까지 누적된 오차를 고려하여 제어량을 생성한다. 이

를 통해 정상상태 오차(Steady-State Error)를 제거할 수 있지만, 적분값이 과도하게 누적될 경우 시스템의 응답이 느려지고 진동이 증가할 수 있다.

미분 제어(D)는 오차의 변화율을 이용하여 제어량을 생성한다. 쇠구슬이 목표 위치를 향해 빠르게 이동하는 경우 이를 미리 감지하여 반대 방향의 제어량을 생성하므로 진동 및 오버슈트를 감소시키는 역할을 수행한다. 또한 시스템의 안정성을 향상시키고 보다 부드러운 제어가 가능하도록 한다.

본 작품에서는 쇠구슬의 위치를 빠르게 복원하면서도 진동을 최소화하기 위하여 P 제어와 D 제어를 적용하였다. 터치패널로 측정된 현재 좌표와 목표 좌표의 차이를 계산하여 PID 제어기의 입력으로 사용하였으며, 계산된 출력값은 플랫폼의 목표 기울기로 변환되어 역기구학 알고리즘에 전달된다. 반복적인 실험을 통하여 최적의 PID 게인을 선정하였으며, 이를 통해 쇠구슬이 플랫폼 중심에서 안정적으로 유지될 수 있도록 하였다.



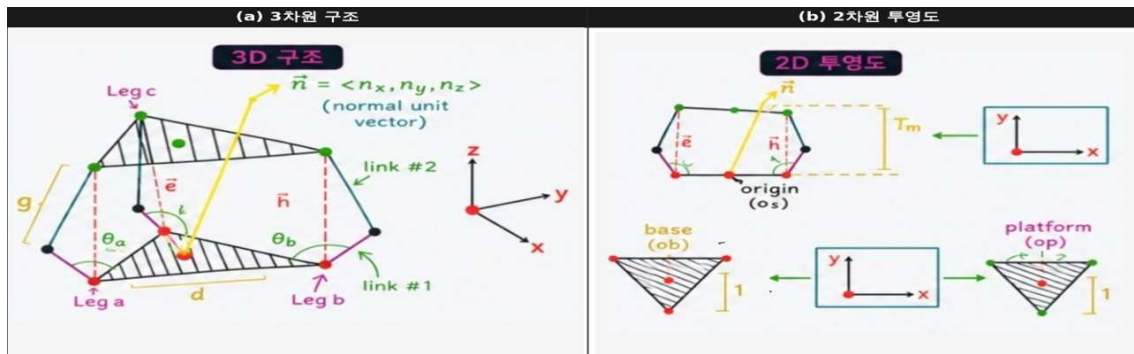
2.2.3 역기구학(Inverse Kinematics)

플랫폼의 자세 제어를 위해서는 PID 제어기가 산출한 목표 기울기를 각 서보모터의 회전 각도로 변환하는 과정이 필요하며, 이를 위해 3RPS 병렬 매니플레이터의 역기구학을 적용하였다. 본 역기구학 수식은 공개 저장소에서 제시한 3RPS 병렬 매니플레이터 역기구학 모델[1]을 참고하여 본 시스템에 적용하였다.

(역기구학 모델[1])

(1) 좌표계 및 변수 정의

3RPS 구조는 베이스(base)와 상단 플랫폼(platform)이 120° 간격으로 배치된 세 개의 다리(Leg a, b, c)로 연결된 병렬 구조이다. 역기구학 계산에 사용되는 변수는



다음과 같다.

- 상수(기구 치수): d, e, g 는 베이스 중심에서 각 Leg 코너까지의 거리, f 와 h 는 각각 Link #1과 Link #2의 길이, l 은 베이스·플랫폼 삼각형 변의 길이이다.
- 입력 변수: 플랫폼의 법선 단위 벡터 $n = (n_x, n_y, n_z)$ 와 플랫폼 높이 T_m 이다.

여기서 입력 변수인 법선 벡터와 높이는 PID 제어기가 출력한 목표 자세(방향·기울기)에 해당한다.

(2) 계산 과정

목표 자세가 주어지면, 먼저 각 Leg의 3차원 좌표를 계산한다. 기준이 되는 Leg a의 좌표는 베이스 중심(OB)에서 Leg a 코너까지의 벡터를 기준으로 산출하며, Leg b와 Leg c는 구조의 $120^\circ \cdot 240^\circ$ 회전 대칭성을 이용하여 Leg a와 동일한 방식으로 계산한다.

각 Leg의 좌표로부터 벡터 길이 $|a|, |b|, |c|$ (베이스에서 플랫폼까지의 거리)를 구한 뒤, 코사인 법칙을 적용하여 각 서보모터의 회전 각도 $\theta_a, \theta_b, \theta_c$ 를 산출한다. 기준이 되는 Leg a의 회전 각도는 다음 식 (2-1)과 같이 계산된다.

$$\theta_a = \cos^{-1}\left(\frac{a_y}{a_x}\right) + \cos^{-1}\left(\frac{a_x^2 + a_z^2 - g^2}{2a_x f}\right) \quad (2-1)$$

Leg b와 Leg c의 회전 각도(θ_b, θ_c)도 동일한 코사인 법칙을 기반으로, 각각 $120^\circ \cdot 240^\circ$ 회전된 좌표계에 대해 같은 방식으로 산출된다.

(3) 계산 흐름 요약

최종적으로 역기구학 알고리즘은 다음 5단계로 동작한다.

1. 목표 플랫폼 자세 입력 (법선 벡터 n , 높이 T_m)
2. 각 Leg의 위치 좌표 계산 — $(a_x, a_y, a_z), (b_x, b_y, b_z), (c_x, c_y, c_z)$
3. 각 Leg의 벡터 길이 계산 — $|a|, |b|, |c|$
4. 코사인 법칙으로 각 모터의 회전 각도 계산 — $\theta_a, \theta_b, \theta_c$

5. 산출된 각도를 각 서보모터에 출력하여 플랫폼 자세 제어

이를 통해 PID 제어기의 출력(목표 기울기)이 실제 서보모터의 물리적 회전 각도로 변환되어, 플랫폼이 목표 자세를 정확히 추종하도록 구현하였다.

2.2.4 MPU6050을 이용한 자세보정

본 작품의 최종 목표는 Ball & Plate 시스템을 이동형 플랫폼에 탑재하여 차량이 이동하는 환경에서도 쇠구슬의 위치를 유지하는 것이다. 그러나 이동형 플랫폼은 가속, 감속, 회전 및 노면 진동에 의해 지속적으로 외란이 발생하며, 이러한 외란은 쇠구슬의 위치를 변화시키는 원인이 된다.

이를 해결하기 위하여 본 작품에서는 MPU6050 관성센서를 이용하여 플랫폼의 자세를 측정하고 보정하는 방법을 적용하였다. MPU6050은 3축 가속도계(Accelerometer)와 3축 자이로스코프(Gyroscope)를 하나의 칩에 통합한 6축 관성센서이다.

가속도계는 중력 방향을 이용하여 플랫폼의 기울기를 계산할 수 있다. 따라서 절대적인 Roll 및 Pitch 값을 얻을 수 있다는 장점이 있지만 진동과 노이즈에 민감하다는 단점이 존재한다.

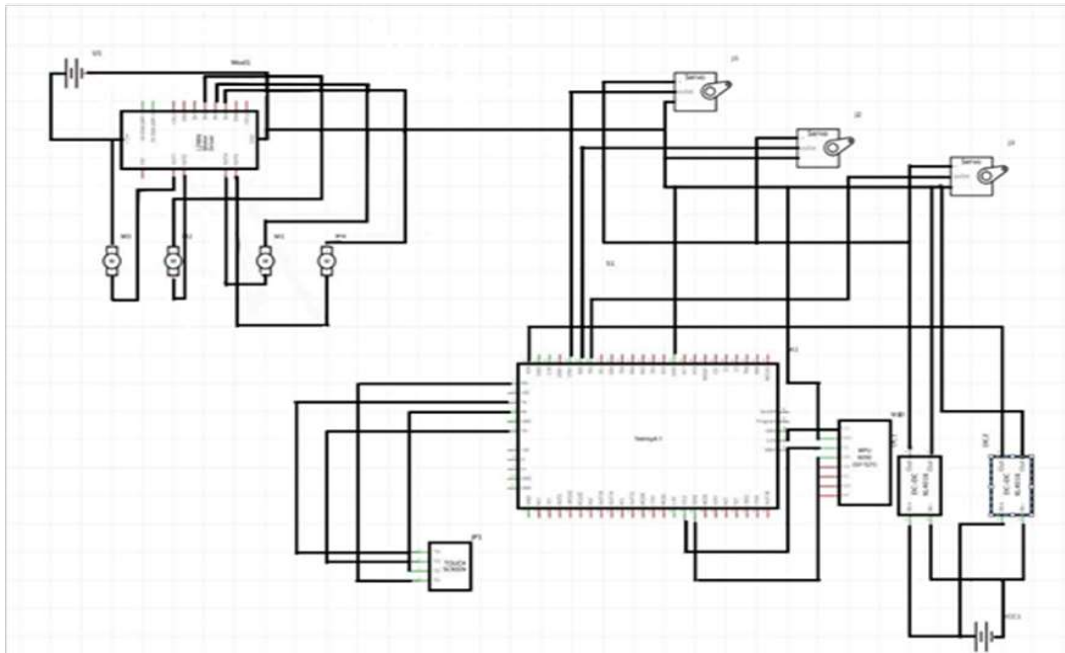
반면 자이로스코프는 각속도를 측정하여 자세 변화를 빠르게 감지할 수 있다. 하지만 각속도를 적분하여 자세를 계산하기 때문에 시간이 지남에 따라 누적 오차(Drift)가 발생한다는 단점이 있다.

본 작품에서는 두 센서의 장점을 결합하기 위하여 상보필터(Complementary Filter)를 적용하였다. 상보필터는 자이로스코프의 빠른 응답성과 가속도계의 안정적인 기준값을 결합하여 보다 정확한 Roll 및 Pitch 값을 계산하는 필터이다.

계산된 Roll 및 Pitch 값은 PID 제어기의 출력값에 반영되어 플랫폼의 기울기를 보정하는 데 사용된다. 예를 들어 차량이 전진하면서 플랫폼이 순간적으로 뒤로 기울어지는 경우 MPU6050은 이를 감지하여 PID 출력에 보상값을 추가하게 된다. 이를 통해 쇠구슬이 한쪽 방향으로 급격하게 이동하는 현상을 줄일 수 있으며 이동 중에도 안정적인 Ball Balancing 성능을 유지할 수 있다.

또한 시스템 초기 구동 시 일정 시간 동안 센서 데이터를 수집하여 기준 오프셋을 계산하고 이를 보정값으로 적용하였다. 이를 통해 센서 장착 오차 및 초기 영점 오차를 최소화하였으며 보다 안정적인 자세 제어가 가능하도록 하였다.

2.2.5 전체 회로도



본 작품의 전체 회로도를 나타낸 것이다. 시스템의 중앙 제어장치로 Teensy 4.1을 사용하였으며, 4선식 저항막 터치패널을 통해 쇠구슬의 위치 정보를 측정한다. 측정된 위치 데이터는 Teensy 4.1에서 처리되며 PID 제어 및 역기구학 연산수행하여 3개의 MG996R 서보모터를 제어한다.

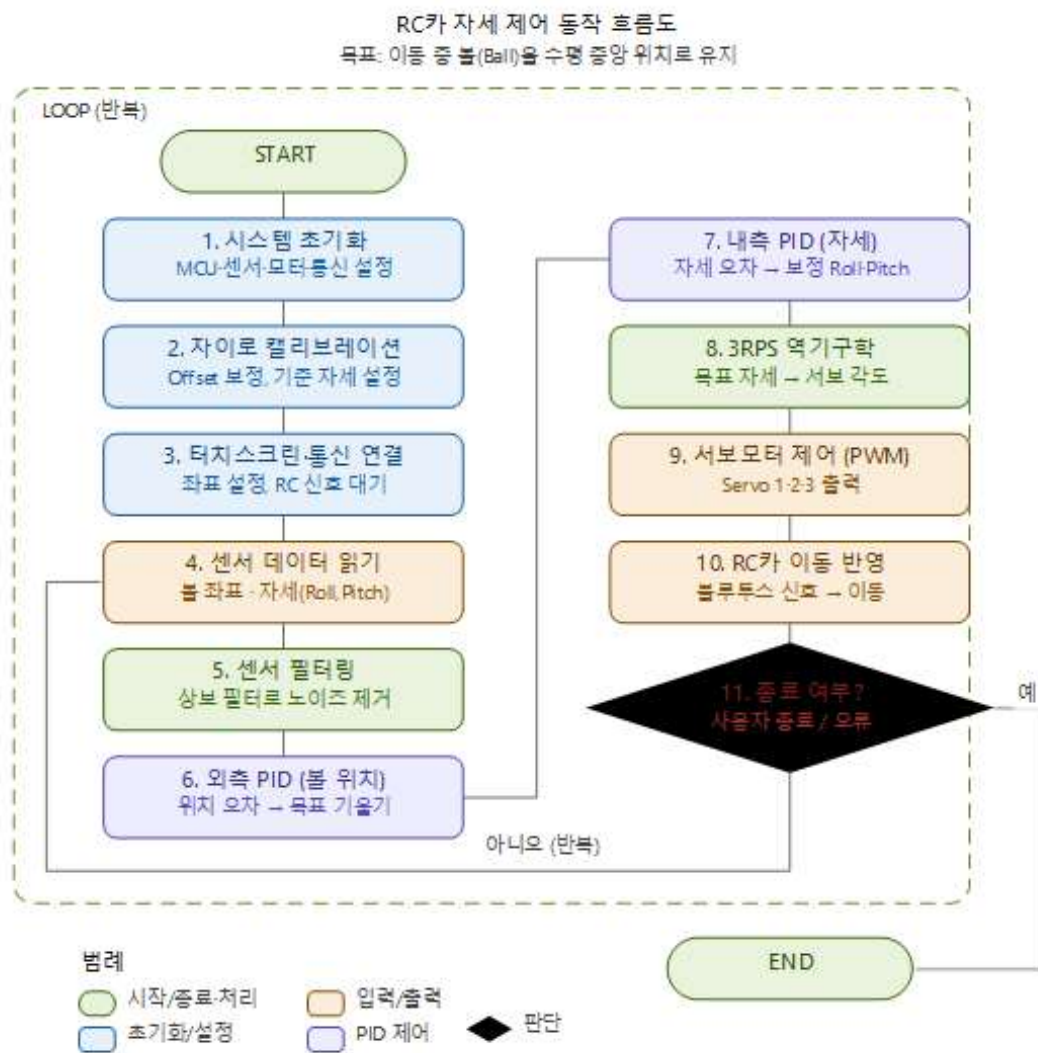
전원은 외부 배터리를 사용하며, 강압기(DC-DC Converter)를 통해 각 장치에 필요한 전압으로 변환하여 공급하였다. 서보모터는 동작 시 큰 전류가 필요하므로 마이크로컨트롤러와 분리된 외부 전원을 사용하였으며, 모든 장치는 공통 GND(Common Ground)로 연결하여 기준 전위를 일치시켰다.

또한 Teensy 4.1은 USB 시리얼 통신을 통해 PC와 연결되며, 센서 데이터와 제어 상태를 실시간으로 모니터링할 수 있도록 구성하였다. 이를 통해 터치패널, 제어기, 서보모터 및 전원부가 하나의 시스템으로 동작하여 쇠구슬의 위치를 안정적으로 제어할 수 있도록 설계하였다.

2.3 구현방법

2.3.1 터치패널을 이용한 공 위치 검출

본 작품에서는 4선식 저항막 터치패널을 이용하여 플랫폼 위에 놓인 쇠구슬의 위치를 실시간으로 측정하였다. 터치패널은 X축과 Y축의 전압값을 ADC로 읽어 좌표를 계산하는 방식으로 동작한다. Teensy 4.1은 터치패널의 X+, X-, Y+, Y- 단자를 이용하여 X축과 Y축 좌표를 순차적으로 측정하며, 측정된 좌표값은 PID 제어기의 입력값으로 사용된다.



터치패널에서 획득한 좌표 데이터는 노이즈 제거를 위하여 필터링 과정을 거친 후 쇠구슬의 현재 위치로 사용된다. 이를 통해 플랫폼 위에서 이동하는 쇠구슬의 위치를

실시간으로 추적할 수 있으며, 목표 위치와의 오차를 계산하여 플랫폼의 기울기를 제어할 수 있다. 아래는 좌표를 측정하는 코드이다.

```
#include <TouchScreen.h>

#define XP 14
#define XM 16
#define YP 15
#define YM 17

TouchScreen ts(XP, YP, XM, YM, 300);

int ballX = 0;
int ballY = 0;

void setup() {
  Serial.begin(115200);
}

void loop() {

  TSPoint p = ts.getPoint();

  pinMode(XM, OUTPUT);
  pinMode(YP, OUTPUT);

  if (p.z > 50) {

    ballX = p.x;
    ballY = p.y;

    Serial.print("X : ");
    Serial.print(ballX);
    Serial.print(" Y : ");
    Serial.println(ballY);
  }

  delay(10);
}
```

터치패널을 통해 쇠구슬의 위치를 측정한 결과, X축과 Y축 좌표가 실시간으로 출력되는 것을 확인할 수 있었다. 측정된 좌표값은 PID 제어기의 입력값으로 사용되며,

목표 위치와의 오차를 계산하여 플랫폼의 기울기를 조절한다. 쇠구슬이 중심 부근에 위치할 경우 X좌표는 약 499, Y좌표는 약 513의 값을 나타내며, 좌표 변화에 따라 서보모터의 각도가 실시간으로 수정되는 것을 확인할 수 있었다.

A terminal window titled 'TERMINAL' displays three columns of data separated by vertical bars. The first column shows coordinates (X,Y) with a red circular icon, the second shows rotation rates (R,P) with a yellow triangle icon, and the third shows motor angles (A,B,C) with a grey gear icon. The data is updated in real-time, showing slight variations in the values.

좌표 (X,Y)	자이로 (R,P)	모터각도 (A,B,C)
499, 513	-0.23, 0.01	91.1, 97.0, 88.7
499, 513	-0.23, 0.01	91.1, 97.0, 88.7
499, 514	-0.23, 0.01	91.1, 97.0, 88.7
499, 513	-0.23, 0.01	91.1, 97.0, 88.7
499, 513	-0.23, 0.01	91.1, 97.0, 88.7
499, 514	-0.23, 0.01	91.1, 97.1, 88.7
499, 513	-0.23, 0.01	91.1, 97.1, 88.7
499, 513	-0.23, 0.01	91.1, 97.1, 88.7
499, 513	-0.23, 0.01	91.1, 97.1, 88.7
499, 513	-0.23, 0.01	91.1, 97.1, 88.7
499, 513	-0.23, 0.01	91.1, 97.1, 88.7
499, 513	-0.23, 0.01	91.1, 97.1, 88.7
499, 513	-0.23, 0.01	91.1, 97.1, 88.7

2.3.2 PID 제어 알고리즘 구현

쇠구슬을 목표 위치에 안정적으로 유지하기 위하여 PID(Proportional-Derivative) 제어 알고리즘을 적용하였다. 터치패널을 통해 측정된 쇠구슬의 현재 위치와 목표 위치 사이의 오차를 계산하고, 이를 이용하여 플랫폼의 기울기를 결정하였다.

PID 제어에서 비례 제어(P)는 현재 위치 오차에 비례하여 제어량을 생성하며, 쇠구슬이 목표 위치에서 멀어질수록 더 큰 보정이 이루어지도록 한다. 또한 미분 제어(D)는 쇠구슬의 이동 속도를 고려하여 과도한 진동과 오버슈트를 감소시키는 역할을 수행한다.

시스템에서는 터치패널로부터 실시간으로 X축과 Y축 좌표를 측정한 후 목표 좌표와의 오차를 계산하였다. 계산된 PID 출력값은 플랫폼의 기울기 값으로 변환되며, 이후 역기구학 알고리즘을 통해 각 서보모터의 회전 각도로 변환된다. 최종적으로 서보모터가 플랫폼의 자세를 조절하여 쇠구슬이 목표 위치를 유지하도록 제어하였다.

PID 제어 알고리즘의 주요 구현 코드를 나타낸다.

```

errorX = TARGET_X - ballX;
errorY = TARGET_Y - ballY;

outputX = Kp * errorX + Kd * (errorX - prevErrorX);
outputY = Kp * errorY + Kd * (errorY - prevErrorY);

prevErrorX = errorX;
prevErrorY = errorY;

```

터치패널로 측정한 현재 좌표와 목표 좌표의 오차를 이용하여 PID 제어값을 계산하고 플랫폼의 기울기를 결정하였다.

– 2.3.3 역기구학 계산 구현

PID 제어를 통해 계산된 플랫폼의 기울기 값을 실제 서보모터의 회전 각도로 변환하기 위하여 역기구학(Inverse Kinematics) 알고리즘을 적용하였다. Ball & Plate 시스템은 3개의 서보모터를 이용하여 플랫폼의 자세를 제어하므로, 목표 기울기에 따라 각 서보모터가 이동해야 하는 각도를 계산할 필요가 있다.

터치패널로부터 측정된 쇠구슬의 위치는 PID 제어를 통해 X축과 Y축 방향의 보정값으로 변환된다. 이후 역기구학 알고리즘은 플랫폼의 목표 기울기와 기구부의 구조적 파라미터를 이용하여 각 서보모터(A, B, C)가 회전해야 하는 각도를 계산한다.

본 시스템에서는 Machine 클래스를 이용하여 역기구학을 구현하였으며, 플랫폼의 높이와 기울기 정보를 입력받아 각 서보모터의 목표 각도를 계산하였다. 계산된 각도는 초기 영점값을 기준으로 보정되며, 각도 제한 및 필터링 과정을 거친 후 서보모터에 전달된다. 이를 통해 플랫폼의 기울기를 정밀하게 조절하여 쇠구슬이 목표 위치를 유지할 수 있도록 하였다.

역기구학 계산을 수행하는 주요 코드를 나타낸다.

```

double da = machine.theta(0, BASE_HEIGHT, -compX, -compY);
double db = machine.theta(1, BASE_HEIGHT, -compX, -compY);
double dc = machine.theta(2, BASE_HEIGHT, -compX, -compY);

float targetA = INIT_ANGLE_A + (da - IK_OFFSET);
float targetB = INIT_ANGLE_B + (db - IK_OFFSET);
float targetC = INIT_ANGLE_C + (dc - IK_OFFSET);

servoA.write(smoothA);
servoB.write(smoothB);
servoC.write(smoothC);

```

PID 제어를 통해 계산된 플랫폼의 기울기 값을 역기구학 알고리즘에 입력하여 각 서보모터의 목표 각도를 계산하고, 이를 통해 플랫폼의 자세를 제어하였다.

- 2.3.4 서보모터 제어

본 작품에서는 플랫폼의 자세를 제어하기 위하여 3개의 MG996R 서보모터를 사용하였다. PID 제어기와 역기구학 알고리즘을 통해 계산된 각 서보모터의 목표 각도는 서보모터 제어 신호로 변환되어 출력된다.

계산된 각도는 기구부의 동작 범위를 고려하여 제한하였으며, 급격한 각도 변화로 인한 진동을 감소시키기 위해 저역통과 필터(Low Pass Filter)를 적용하였다. 필터링된 각도값은 서보모터에 전달되어 플랫폼의 기울기를 부드럽게 조절하도록 구현하였다. 서보모터 제어를 수행하는 주요 코드를 나타낸다.

```
smoothA = (smoothA * (1.0 - FILTER_VAL)) + (targetA * FILTER_VAL);
smoothB = (smoothB * (1.0 - FILTER_VAL)) + (targetB * FILTER_VAL);
smoothC = (smoothC * (1.0 - FILTER_VAL)) + (targetC * FILTER_VAL);

servoA.write(smoothA);
servoB.write(smoothB);
servoC.write(smoothC);
```

– 2.3.5 MPU6050 수평 보정

본 작품에서는 이동형 플랫폼 주행 시 발생하는 진동과 기울기에 의한 영향을 보정하기 위하여 MPU6050 관성센서를 사용하였다. MPU6050은 3축 가속도계와 3축 자이로스코프를 내장하고 있으며, 이를 이용하여 플랫폼의 기울기 정보를 측정할 수 있다.

가속도계와 자이로스코프 데이터는 상보필터(Complementary Filter)를 적용하여 Roll 및 Pitch 각도를 계산하였다. 계산된 기울기 값은 PID 제어 출력에 반영되어 플랫폼의 자세를 보정함으로써 이동 중에도 쇠구슬을 안정적으로 유지할 수 있도록 하였다. MPU6050을 이용한 자세 보정 코드를 나타낸다.

```
gyroPitch = 0.98 * (gyroPitch + gyroRateY * dt)
            + 0.02 * accelPitch;

gyroRoll = 0.98 * (gyroRoll + gyroRateX * dt)
            + 0.02 * accelRoll;

compX = outputX - ((gyroRoll + 0.24) * 0.4);
compY = outputY - ((gyroPitch + 0.03) * 0.4);
```

– 2.3.6 Ball & Plate 제어 알고리즘 구현

본 작품의 Ball & Plate 시스템은 터치패널, 자이로 센서(MPU6050), PID 제어기, 역기구학 알고리즘 및 서보모터 제어를 통합하여 구현하였다. 먼저 터치패널을 통해 쇠구슬의 현재 위치를 측정하고 목표 위치와의 오차를 계산한다.

이와 동시에 자이로 센서를 이용하여 플랫폼의 기울기(Roll, Pitch)를 실시간으로 측정한다. 본 시스템은 이동형 플랫폼 위에서 동작하므로, 주행 중 발생하는 차체의 흔들림과 진동이 외란으로 작용한다. 자이로 센서는 이러한 차체 자세 변화를 감지하여, 플랫폼이 외란에도 수평 기준을 유지하도록 보정하는 데 활용된다. 이때 가속도계와 자이로의 측정값은 상보 필터(Complementary Filter)를 통해 결합하여 노이즈를 제거하고 안정적인 자세 데이터를 확보한다.

계산된 위치 오차는 외측 PID 제어기를 통해 플랫폼의 목표 기울기 값으로 변환되고, 자이로 센서로 측정한 자세 오차는 내측 PID 제어기를 통해 보정되어 최종 기울기 값이 결정된다. 즉, 공의 위치를 제어하는 외측 루프와 플랫폼의 자세를 안정화하는 내측 루프가 결합된 이중 루프 구조로 동작한다. 이렇게 결정된 최종 목표 기울기는 역기구학 알고리즘을 이용하여 각 서보모터의 회전 각도로 변환되며, 서보모터가 플랫폼의 자세를 조절하여 쇠구슬이 목표 위치를 유지하도록 제어한다.

이러한 과정을 반복 수행함으로써, 차체가 이동하거나 외란이 발생하는 상황에서도 플랫폼의 수평을 유지하는 동시에 쇠구슬의 위치를 실시간으로 추적하여 안정적으로 균형을 유지할 수 있도록 구현하였다.

- 2.3.7 이동형 플랫폼 적용

본 작품에서는 기존 Ball & Plate 시스템을 이동형 플랫폼에 탑재하여 주행 중에도 쇠구슬의 위치를 유지할 수 있도록 구현하였다. 이동형 플랫폼은 L298N 모터 드라이버와 DC 모터를 이용하여 구동되며, 블루투스 통신을 통해 사용자의 주행 명령을 수신한다.

스마트폰에서 전송된 전진(F), 후진(B), 좌회전(L), 우회전(R), 정지(S) 명령은 블루투스 모듈을 통해 Teensy 4.1로 전달되며, 이를 바탕으로 이동형 플랫폼을 제어한다. 또한 급격한 속도 변화로 인한 쇠구슬의 이탈을 방지하기 위하여 가감속 제어 알고리즘을 적용하여 부드러운 주행이 가능하도록 구현하였다.

이를 통해 이동 중 발생하는 외란에도 쇠구슬의 위치를 유지할 수 있는 이동형 Ball & Plate 시스템을 구현하였다.

2.4 주요기능 실험

- 2.4.1 터치패널 위치 검출 실험

본 실험에서는 4선식 저항막 방식 터치패널을 이용하여 공의 위치를 정확하게 검출할 수 있는지 확인하였다. 터치패널은 X축과 Y축 방향의 전압 분배 원리를 이용하여 접촉 위치를 좌표값으로 변환하며, 본 시스템에서는 측정된 좌표값을 실시간으로 PID 제어기의 입력값으로 사용한다.

위치 검출 성능을 확인하기 위하여 터치패널의 중심부, 상단부 및 하단부를 각각 터치한 후 출력되는 좌표값을 측정하였다. 실험 결과 중심부에서는 X좌표 약 499, Y좌표 약 513이 출력되어 터치패널의 중앙 영역이 정상적으로 검출됨을 확인하였다. 또한 상단 영역에서는 X좌표 약 57, Y좌표 약 508이 측정되었으며, 하단 영역에서는 X좌표 약 951, Y좌표 약 515가 측정되었다.

실험 결과를 통해 터치 위치가 상단에서 하단으로 이동함에 따라 X축 좌표값이 약 57에서 951까지 연속적으로 변화하는 것을 확인하였다. 이는 터치패널이 위치 변화에 따라 안정적으로 좌표값을 출력하고 있음을 의미한다. 또한 Y축 좌표는 측정 위치에 따라 상대적으로 작은 변화만 나타내었는데, 이는 본 실험이 주로 상하 방향의 위치 변화에 초점을 맞추어 수행되었기 때문이다.

각 위치에서 반복 측정을 수행한 결과 좌표값의 변화 폭이 크지 않았으며 측정값이 일정 범위 내에서 유지되는 것을 확인하였다. 이를 통해 터치패널이 높은 반복성과 안정성을 가지며 외부 노이즈의 영향 없이 위치 정보를 측정할 수 있음을 확인하였다.

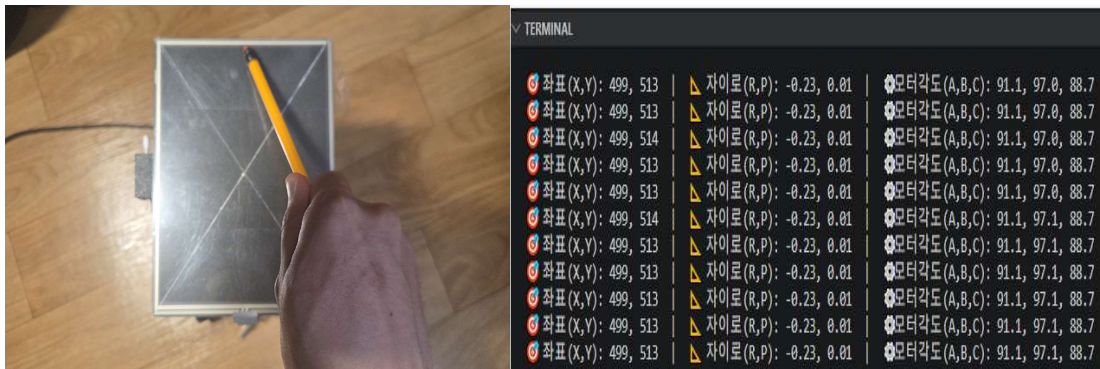
또한 측정된 좌표값은 실시간으로 Teensy 4.1 제어기에 전달되어 목표 위치와 현재 위치의 오차를 계산하는 데 사용되었다. PID 제어기는 해당 오차를 기반으로 플랫폼의 기울기를 계산하였으며, 역기구학 알고리즘을 통해 각 서보모터의 회전 각도를 산출하였다. 실험 과정에서 터치 위치를 변경할 때마다 플랫폼이 해당 방향으로 기울어지는 것을 확인할 수 있었으며, 이를 통해 터치패널의 위치 검출 기능이 제어 시스템과 정상적으로 연동됨을 검증하였다.

따라서 본 실험을 통해 4선식 저항막 터치패널이 공의 위치를 안정적으로 검출할 수 있으며, 측정된 좌표값을 실시간 위치 제어에 활용할 수 있음을 확인하였다.

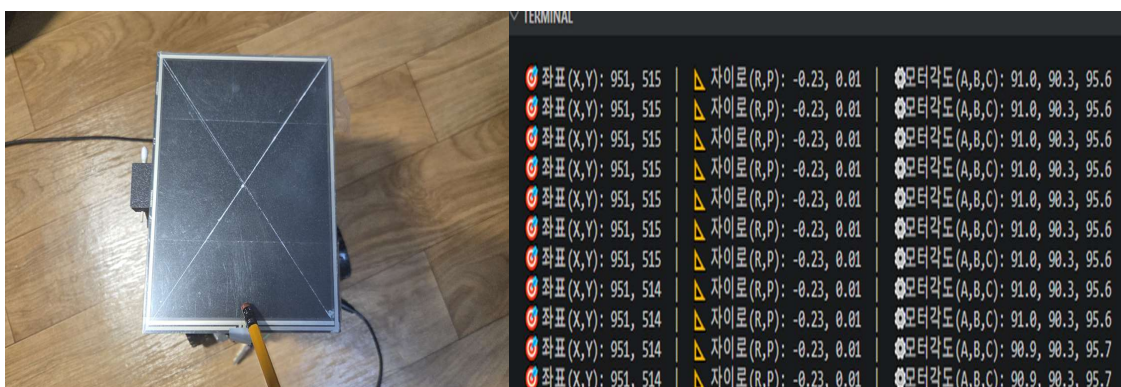
1. 중심 부근에 압력을 가했을 때 x축 y축 좌표값



2. 상단 부근에 압력을 가했을 때 x축 y축 좌표값



3. 하단 부근에 압력을 가했을 때 x축 y축 좌표값



- 2.4.2 역기구학 및 서보모터 동작 실험

본 실험에서는 PID 제어기를 통해 계산된 플랫폼의 목표 기울기가 역기구학 알고리즘을 통해 각 서보모터의 회전 각도로 올바르게 변환되는지 확인하였다.

Ball Balancing System은 3개의 서보모터가 120° 간격으로 배치된 3RPS 구조를 사용한다. 따라서 플랫폼을 특정 방향으로 기울이기 위해서는 각 서보모터가 서로 다른 각도로 회전해야 하며, 이를 계산하기 위해 역기구학 알고리즘이 필요하다.

실험 과정에서는 X축 및 Y축 방향으로 목표 기울기를 인가한 후 각 서보모터의 동작을 관찰하였다. 실험 결과 플랫폼이 목표 방향으로 정상적으로 기울어졌으며, 각 서보모터가 독립적으로 회전하면서도 전체 플랫폼은 하나의 평면을 유지하는 것을 확인하였다.

또한 목표 기울기 값을 변경할 때마다 서보모터의 회전 각도가 실시간으로 변화하였으며, 플랫폼 역시 이에 대응하여 부드럽게 자세를 변경하였다. 특정 방향으로 기울기를 명령한 경우 해당 방향의 서보모터는 상승 또는 하강 동작을 수행하였으며, 나머지 서보모터는 이에 연동하여 플랫폼의 균형을 유지하였다.

반복 실험 결과 역기구학 알고리즘을 통해 계산된 각도가 실제 플랫폼의 움직임과 일치함을 확인하였으며, 서보모터 간의 동작 오차 없이 안정적으로 자세 제어가 수행되는 것을 검증하였다. 따라서 본 실험을 통해 역기구학 알고리즘과 서보모터 구동 시스템이 정상적으로 동작함을 확인하였다.

[\(역기구학 및 서보모터 동작 실험 영상\)](#)

- 2.4.3 PID 게인 튜닝 실험

Ball Balancing System의 안정적인 제어를 위하여 PID 게인 값 변화에 따른 쇠구슬의 응답 특성을 비교 분석하였다. 본 작품에서는 적분 제어(I)를 사용하지 않고 비례 제어(P)와 미분 제어(D)를 적용한 PD 제어기를 사용하였다. 적분 제어는 정상 상태 오차를 줄일 수 있지만 Ball Balancing System과 같이 빠른 응답성이 요구되는 시스템에서는 적분 누적으로 인한 진동이 발생할 수 있으므로 적용하지 않았다.

비례 게인(K_p)은 현재 오차에 비례하여 제어량을 결정한다. K_p 값을 낮게 설정한 경우 쇠구슬이 목표 위치까지 이동하는 속도가 느려지고 외란 발생 시 중심 위치로 복귀하는 시간이 길어지는 현상이 나타났다. 반면 K_p 값을 증가시키면 응답 속도는 향상되었으나, 과도하게 증가할 경우 쇠구슬이 목표 위치를 지나치는 오버슈트(Overshoot)가 발생하고 지속적인 진동이 나타났다.

미분 게인(K_d)은 오차의 변화율을 이용하여 제어 신호를 보정하는 역할을 수행한다. K_d 값을 적용하지 않은 경우 목표 위치 부근에서 쇠구슬이 반복적으로 왕복 운동하며 진동하는 현상이 발생하였다. 반면 K_d 값을 증가시키면 오버슈트가 감소하고 진동이 억제되어 보다 안정적인 응답 특성을 얻을 수 있었다. 그러나 K_d 값이 지나치게 커질 경우 시스템의 반응이 둔화되어 쇠구슬의 이동 속도가 감소하는 현상이 확인되었다.

반복적인 실험을 통하여 다양한 게인 값을 비교한 결과, $K_p=0.0003$ 및 $K_d=0.023$ 일 때 응답 속도와 안정성 측면에서 가장 우수한 성능을 나타내었다. 해당 게인 값에서는 쇠구슬이 목표 위치에 빠르게 도달하면서도 과도한 오버슈트 없이 안정적으로 중심을 유지하였으며, 외란이 발생하더라도 신속하게 목표 위치로 복귀하는 것을 확인할 수 있었다. 따라서 본 작품에서는 $K_p=0.0003$, $K_d=0.023$ 을 최종 제어 게인으로 선정하여 시스템에 적용하였다.

(PID 게인 튜닝 실험 영상)

- 2.4.4 MPU6050 자세 보정 실험

본 실험에서는 MPU6050 관성 센서를 이용하여 이동 플랫폼의 자세 변화를 측정하고, 이를 제어기에 반영하여 플랫폼의 수평 상태를 유지할 수 있는지 확인하였다.

기존 Ball Balancing System은 대부분 고정된 환경에서 동작하기 때문에 플랫폼 자체의 자세 변화가 거의 발생하지 않는다. 그러나 본 작품은 RC카 기반 이동 플랫폼 위에 Ball Balancing System을 탑재하였기 때문에 주행 중 발생하는 가속, 감속, 회전 및 외부 충격에 의해 플랫폼의 기울기가 지속적으로 변화한다. 이러한 자세 변화는 공의 위치 제어 성능을 저하시킬 수 있으므로 이를 실시간으로 측정하고 보정할 필요가 있다.

MPU6050은 3축 가속도 센서와 3축 자이로스코프를 내장한 관성 센서로, 플랫폼의 기울기와 회전 정보를 측정할 수 있다. 본 시스템에서는 가속도 센서와 자이로스코프 데이터를 상보필터(Complementary Filter)를 이용하여 융합하였으며, 이를 통해 Roll 및 Pitch 각도를 계산하였다. 상보필터를 적용함으로써 가속도 센서의 노이즈와 자이로스코프의 누적 오차를 효과적으로 보완할 수 있었다.

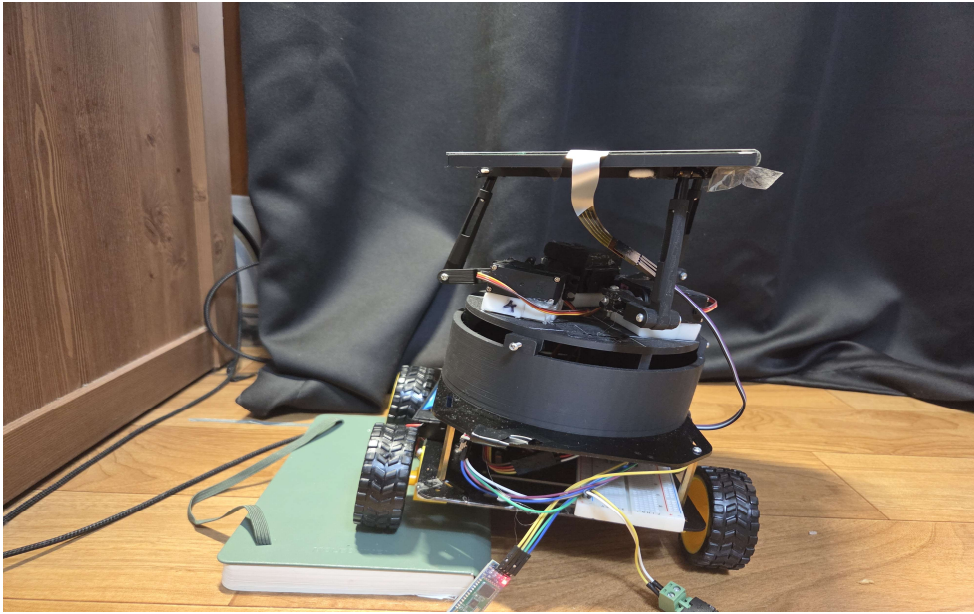
실험은 RC카를 전방 및 측면 방향으로 기울인 상태에서 수행하였다. 실험 결과 차량이 전방으로 기울어진 경우 MPU6050이 Pitch 방향의 자세 변화를 정상적으로 검출하였으며, 측면으로 기울어진 경우에는 Roll 방향의 자세 변화를 안정적으로 측정하는 것을 확인하였다. 또한 기울기의 방향과 크기에 따라 센서 출력값이 실시간으로 변화하는 것을 확인할 수 있었다.

측정된 Roll 및 Pitch 값은 PID 제어기의 보정값으로 사용되었다. 플랫폼이 특정 방향으로 기울어질 경우 제어기는 해당 방향의 오차를 계산하여 역기구학 알고리즘에 반영하였으며, 각 서보모터의 회전 각도를 조절하여 플랫폼이 다시 수평 상태를 유지하도록 하였다. 이를 통해 차량의 움직임이나 외부 충격이 발생하더라도 플랫폼의 자세 변화를 최소화할 수 있었다.

반복 실험 결과 MPU6050 센서는 이동 환경에서 발생하는 자세 변화를 안정적으로 검출하였으며, PID 제어기와 연동하여 플랫폼의 수평 상태 유지에 효과적으로 활용될 수 있음을 확인하였다. 따라서 본 실험을 통해 MPU6050 기반 자세 보정 시스템이 이동형 Ball Balancing System의 안정성 향상에 기여함을 검증하였다.



(차체를 앞으로 기울인 시점)



(차체를 옆으로 기울인 시점)

- 2.4.5 Ball Balancing 실험

본 실험에서는 터치패널을 이용한 위치 검출, PID 제어기, 역기구학 알고리즘 및 서보모터 구동 시스템을 통합하여 쇠구슬의 위치를 안정적으로 제어할 수 있는지 확인하였다. 실험에 사용된 쇠구슬은 플랫폼 위에 배치하였으며, 목표 위치는 터치패널 중심 좌표인 $X=499$, $Y=513$ 부근으로 설정하였다.

실험 과정에서 터치패널은 쇠구슬의 현재 위치를 실시간으로 측정하여 제어기에 전달하였다. PID 제어기는 목표 위치와 현재 위치의 오차를 계산하고, 계산된 제어량을 플랫폼의 목표 기울기로 변환하였다. 이후 역기구학 알고리즘을 통해 각 서보모터의 회전 각도를 산출하여 플랫폼의 자세를 조절하였다.

초기 실험에서는 쇠구슬이 목표 위치에 접근하는 과정에서 오버슈트와 진동 현상이 발생하였다. 이는 비례 게인(K_p)에 의해 플랫폼이 과도하게 기울어지면서 쇠구슬이 목표 위치를 지나치는 현상 때문이었다. 또한 미분 게인(K_d)이 충분하지 않을 경우 쇠구슬이 중심 부근에서 반복적으로 왕복 운동하며 안정적으로 정지하지 못하는 모습을 확인할 수 있었다.

이러한 문제를 해결하기 위해 PID 게인 튜닝을 수행하였다. 반복 실험을 통해 $K_p=0.0003$, $K_d=0.023$ 을 최적 게인으로 선정하였으며, 이를 적용한 결과 쇠구슬이 목표 위치에 빠르게 도달하면서도 과도한 오버슈트 없이 안정적으로 중심을 유지하는 것을 확인할 수 있었다.

또한 플랫폼에 외부 자극을 가하여 쇠구슬을 의도적으로 이동시키는 실험을 수행하였다. 실험 결과 쇠구슬이 중심 위치에서 벗어나더라도 제어기가 오차를 검출하여 플랫폼의 기울기를 자동으로 조절하였으며, 쇠구슬은 다시 목표 위치로 복귀하는 모습을 확인하였다. 이를 통해 시스템이 외란에 대해서도 안정적인 복원 성능을 가지고 있음을 확인하였다.

추가적으로 RC카 기반 이동 플랫폼 환경에서도 실험을 수행하였다. 차량의 이동 과정에서 발생하는 진동과 기울기 변화는 MPU6050 센서를 통해 측정되었으며, 측정된 자세 정보는 PID 제어기의 보정값으로 활용되었다. 이를 통해 차량이 이동하거나 외부 충격이 발생하더라도 플랫폼의 자세 변화가 감소하였으며 쇠구슬 역시 중심 부근을 유지하는 것을 확인하였다.

실험 결과 Ball Balancing System은 쇠구슬의 위치를 실시간으로 검출하고 목표 위치로 안정적으로 복귀시킬 수 있었으며, 외란이 존재하는 환경에서도 우수한 위치 제어 성능을 나타내었다. 따라서 본 작품에서 구현한 터치패널 기반 위치 검출, PD 제어기, 역기구학 알고리즘 및 자세 보정 시스템이 효과적으로 동작함을 검증하였다.

(Ball Balancing 실험 영상)

- 2.4.6 외란 복원 실험

본 실험에서는 Ball Balancing System에 외부 힘을 가하여 쇠구슬의 위치를 의도적으로 변화시킨 후, 제어 시스템이 목표 위치로 복원할 수 있는지 확인하였다. 이를 통해 PID 제어기와 역기구학 알고리즘이 외란에 대해 얼마나 안정적으로 대응하는지 평가하고자 하였다.

실험은 쇠구슬이 플랫폼 중심 부근에서 안정적으로 유지되는 상태에서 시작하였다. 이후 손을 이용하여 쇠구슬을 특정 방향으로 밀어 외란을 발생시켰으며, 쇠구슬이 목표 위치에서 벗어난 상황을 인위적으로 만들었다. 이때 터치패널은 쇠구슬의 새로운 위치를 실시간으로 측정하여 제어기에 전달하였다.

쇠구슬이 중심 위치에서 벗어나면 PID 제어기는 목표 위치와 현재 위치 간의 오차를 계산하였다. 계산된 오차는 플랫폼의 목표 기울기로 변환되었으며, 역기구학 알고리즘을 통해 각 서보모터의 회전 각도가 산출되었다. 이후 서보모터는 플랫폼을 적절한 방향으로 기울여 쇠구슬이 다시 목표 위치로 이동하도록 유도하였다.

실험 결과 쇠구슬은 외란에 의해 일시적으로 중심 위치에서 벗어났으나, 제어기의 동작에 의해 다시 목표 위치로 복귀하는 것을 확인할 수 있었다. 외란의 크기가 작은 경우에는 빠른 시간 내에 중심 위치로 복원되었으며, 비교적 큰 외란이 가해진 경우에는 복원 시간이 다소 증가하였으나 최종적으로는 안정적으로 목표 위치를 회복하였

다.

또한 복원 과정에서 PD 제어기의 미분 게인(K_d)이 진동을 억제하는 역할을 수행하여 과도한 오버슈트 없이 안정적으로 수렴하는 것을 확인하였다. 비례 게인(K_p)은 복원 속도를 향상시키는 역할을 수행하였으며, 두 게인의 적절한 조합을 통해 빠른 응답성과 안정성을 동시에 확보할 수 있었다.

반복 실험 결과 외란의 방향이 달라지더라도 시스템은 지속적으로 오차를 검출하고 플랫폼의 자세를 조절하여 쇠구슬을 목표 위치로 복귀시킬 수 있었다. 이를 통해 본 작품에서 구현한 PID 제어기와 역기구학 알고리즘이 외부 환경 변화에 대해 안정적으로 대응할 수 있음을 검증하였다.

따라서 본 실험을 통해 Ball Balancing System이 단순히 쇠구슬을 중심에 유지하는 것뿐만 아니라 외부 충격이나 예기치 않은 환경 변화가 발생하더라도 목표 위치를 유지할 수 있는 우수한 외란 복원 성능을 보유하고 있음을 확인하였다.

(외란 복원 실험 영상)

- 2.4.7 이동형 플랫폼 실험

Ball & Plate 시스템을 이동형 플랫폼에 탑재한 후 전진, 후진 및 방향 전환 실험을 수행하였다. 주행 중 차량의 가속 및 감속에 의해 쇠구슬이 이동하는 현상이 발생하였으나 PID 제어와 MPU6050 자세 보정을 통해 쇠구슬의 위치를 유지할 수 있었다. 이를 통해 이동 환경에서도 Ball Balancing 기능이 동작함을 확인하였다.

(이동형 플랫폼 실험 영상)

- 2.4.8 최종 통합 테스트

본 실험에서는 터치패널, PD 제어기, 역기구학 알고리즘, MPU6050 자세 보정 시스템, 서보모터 구동부 및 이동형 RC카 플랫폼을 모두 통합하여 시스템 전체의 동작 성능을 검증하였다. 최종 통합 테스트의 목적은 각 모듈이 독립적으로 정상 동작하는 것을 확인하는 것뿐만 아니라, 모든 구성 요소가 하나의 시스템으로 연동되어 이동 환경에서도 안정적인 Ball Balancing 기능을 수행할 수 있는지 평가하는 데 있다.

실험 과정에서는 쇠구슬을 플랫폼 위에 올려놓은 상태에서 시스템을 동작시켰다. 터치패널은 쇠구슬의 위치를 실시간으로 검출하였으며, 측정된 좌표값은 Teensy 4.1 제어기로 전달되었다. 제어기는 목표 위치와 현재 위치 간의 오차를 계산한 후 PD

제어 알고리즘을 이용하여 플랫폼의 목표 기울기를 결정하였다. 이후 역기구학 알고리즘을 통해 계산된 각 서보모터의 회전 각도를 적용하여 플랫폼의 자세를 조절하였다.

또한 이동 플랫폼이 주행하는 동안 발생하는 기울기 변화와 진동은 MPU6050 센서를 통해 실시간으로 측정되었다. 측정된 Roll 및 Pitch 데이터는 상보필터를 통해 처리되었으며, 플랫폼 자세 보정에 활용되었다. 이를 통해 차량의 가속, 감속 및 방향 전환 과정에서 발생하는 외란이 플랫폼에 미치는 영향을 최소화할 수 있었다.

실험 결과 시스템 초기 구동 시 쇠구슬은 일시적으로 중심 위치에서 벗어나는 모습을 보였으나, 제어가 오차를 검출하고 플랫폼의 기울기를 조절함에 따라 점차 목표 위치로 수렴하는 것을 확인하였다. 또한 쇠구슬이 중심 위치에서 벗어났을 경우에도 플랫폼이 자동으로 기울기를 조절하여 다시 목표 위치로 복귀시키는 모습을 확인할 수 있었다.

이동 플랫폼 주행 실험에서도 모든 모듈이 정상적으로 연동되는 것을 확인하였다. 차량이 직진하거나 방향을 전환하는 과정에서 발생하는 진동과 기울기 변화에도 불구하고 MPU6050 기반 자세 보정 기능이 동작하여 플랫폼의 수평 상태를 유지하였으며, 쇠구슬 역시 중심 부근에서 안정적으로 유지되었다.

특히 실제 이동 환경을 모사하기 위하여 약 1cm 높이의 방지턱을 설치한 후 차량이 이를 통과하도록 실험을 수행하였다. 방지턱 통과 시 차량에는 순간적인 충격과 진동이 발생하였으나 MPU6050 센서가 자세 변화를 실시간으로 감지하였고, PD 제어기와 역기구학 알고리즘이 즉시 반응하여 플랫폼의 자세를 보정하였다. 그 결과 쇠구슬은 순간적으로 미세한 위치 변화를 보였지만 플랫폼 밖으로 이탈하지 않았으며, 짧은 시간 내에 다시 중심 부근으로 복귀하는 것을 확인하였다.

또한 외부 충격이나 인위적인 외란을 가한 경우에도 시스템은 지속적으로 위치 오차를 계산하여 쇠구슬을 목표 위치로 복귀시키는 것을 확인하였다. 이를 통해 본 시스템이 단순한 정적 환경뿐만 아니라 실제 이동 환경에서 발생할 수 있는 외란에도 안정적으로 대응할 수 있음을 검증하였다.

최종적으로 터치패널을 이용한 위치 검출, PD 제어를 이용한 위치 제어, 역기구학 기반 자세 제어, MPU6050을 이용한 자세 보정 및 이동형 플랫폼 환경이 하나의 시스템으로 통합되어 안정적으로 동작함을 검증하였다. 이를 통해 본 작품의 목표인 "이동 환경에서도 쇠구슬의 위치를 안정적으로 유지할 수 있는 Ball Balancing System 구현"을 성공적으로 달성하였음을 확인하였다.

또한 실험 과정에서 터치패널의 측정 노이즈와 서보모터의 기계적 오차로 인해 미세한 진동이 발생하였으나 전체적인 제어 성능에는 큰 영향을 주지 않았다. 향후 칼만 필터(Kalman Filter), 상태 공간 제어(State Space Control) 및 인공지능 기반

제어 알고리즘을 적용한다면 보다 향상된 제어 성능을 기대할 수 있을 것으로 판단된다.

[\(최종 통합 테스트 영상\)](#)

결론

3.1 결과 및 고찰

본 작품에서는 운반 과정에서 발생하는 슬로싱(sloshing)과 같은 외란에 대응하기 위한 능동 자세 제어 기술의 가능성을 검증하기 위하여, Stewart Platform을 단순화한 3RPS 병렬 매니플레이터 구조 기반의 이동형 Ball Balancing System을 설계 및 제작하였다. 점성 감쇠가 거의 없어 제어가 어려운 쇠구슬을 자유 표면 물체의 대표 모델로 선정하였으며, 이를 이동 환경에서도 안정적으로 제어할 수 있는 시스템 구현을 목표로 하였다.

시스템은 4-Wire Resistive Touch Panel을 이용하여 쇠구슬의 위치를 실시간으로 측정하고, PD 제어기와 역기구학 알고리즘을 통해 플랫폼의 목표 기울기를 계산하여 세 개의 서보모터를 제어하도록 설계하였다. 또한 MPU6050 관성센서를 이용하여 이동 플랫폼의 Roll 및 Pitch 정보를 측정하고, 이를 자세 보정에 활용함으로써 이동 중 발생하는 외란에 대응할 수 있도록 구성하였다. 이동 플랫폼은 블루투스 통신을 통해 원격으로 제어할 수 있도록 구현하여 실제 운반 환경을 모사하였다.

실험 결과 터치패널을 이용한 쇠구슬 위치 검출이 안정적으로 수행되었으며, PD 제어기와 역기구학 알고리즘을 통해 쇠구슬을 목표 위치인 중심 부근으로 이동시키고 유지할 수 있음을 확인하였다. 또한 MPU6050 기반 자세 보정을 적용함으로써 이동 중 발생하는 기울기 변화와 진동의 영향을 감소시킬 수 있었으며, 외란이 발생하더라도 쇠구슬을 다시 목표 위치로 복귀시키는 외란 복원 성능을 확인하였다.

특히 이동 환경 검증을 위해 약 1cm 높이의 방지턱을 설치하고 이동 플랫폼이 이를 통과하는 실험을 수행하였다. 방지턱 통과 시 순간적인 충격과 기울기 변화가 발생하였음에도 불구하고 MPU6050 센서가 자세 변화를 실시간으로 검출하고, PD 제어기와 자세 보정 알고리즘이 이를 보상함으로써 쇠구슬이 플랫폼 밖으로 이탈하지 않고 중심 부근을 유지하는 것을 확인하였다. 이를 통해 본 시스템이 단순한 정지 환경뿐만 아니라 실제 이동 환경에서도 안정적으로 동작할 수 있음을 검증하였다.

본 작품은 액체 슬로싱을 직접 제어한 시스템은 아니지만, 슬로싱 억제에 필요한 핵심 요소인 위치 검출, 자세 추정, 외란 보상 및 능동 자세 제어 기술을 통합적으로 구현하였다는 점에서 의의가 있다. 특히 자유 표면 물체보다 제어가 어려운 한계 안정 시스템인 쇠구슬을 이동 환경에서 안정적으로 제어함으로써, 향후 다양한 적재물

안정화 기술에 적용 가능한 기반 기술임을 확인하였다.

또한 이동 플랫폼이 약 1cm 높이의 방지턱을 통과하는 환경에서도 쇠구슬의 균형을 유지할 수 있음을 검증함으로써, 실제 산업 환경에서 발생하는 진동과 충격에 대한 대응 가능성을 확인하였다. 이는 단순한 정지형 Ball Balancing System을 넘어 이동 중 발생하는 외란까지 고려한 능동 자세 제어 시스템이라는 점에서 차별성을 가진다.

향후 본 기술은 제약 및 화학 산업에서의 시약·액체 운반, 반도체 및 디스플레이 공정에서의 정밀 부품 이송, 물류 자동화 시스템의 고정밀 운반 장치 등 다양한 분야에 적용될 수 있을 것으로 기대된다. 또한 자율주행 물류 로봇(AMR), 무인 운반 차량(AGV), 서비스 로봇과 같은 이동형 로봇 시스템에 적용하여 이동 중에도 적재물의 안정성을 유지하는 기술로 확장될 수 있다.

따라서 본 작품은 단순한 교육용 제어 실험 장치를 구현한 것에 그치지 않고, 실제 산업 현장에서 요구되는 적재물 안정화 및 능동 자세 보정 기술의 가능성을 검증하였다는 점에서 의미가 있으며, 향후 스마트 팩토리 및 첨단 자동화 산업에 활용될 수 있는 기초 연구로서의 가치를 가진다.

참고 문헌

B. Brown, 「Ball Balancer : 8 Steps (with Pictures)」, Instructables, 2021..

Tower Pro, 「MG996R Servo Motor Specification」, Tower Pro, 2024.

Paul Stoffregen, 「Teensy 4.1 Development Board Technical Specifications」, PJRC, 2025.

InvenSense, 「MPU-6000 and MPU-6050 Product Specification」, InvenSense Inc., 2013.

Aaed Musa, 「Ball-Balancer-V2」, GitHub Repository,
<https://github.com/aaedmusa/Ball-Balancer-V2>

A. M. Hussien, S. Fathollahi Dehkordi, A. Naeimifard, "Sloshing Suppression in Liquid Transport Systems Using Fuzzy Logic Control Algorithm," Arabian Journal for Science and Engineering, 2025.

A. K. Shakya 외, "Spill-free liquid container handling using deep

reinforcement learning agents in feedback control," Applied Intelligence, vol. 56, 2026.

부록

A. main.cpp 통합코드

```
#include <Arduino.h>
#include <Servo.h>
#include <Wire.h>
#include "InverseKinematics.h"

// ===== [1] 기구부 및 핀 설정 =====
Machine machine(1.6535, 3.125, 1.9685, 3.5433);
const int XP = 14, XM = 16, YP = 15, YM = 17;

Servo servoA, servoB, servoC;
const int SERVO_PIN_A = 4;
const int SERVO_PIN_B = 3;
const int SERVO_PIN_C = 5;

// ===== [2] 영점 및 캘리브레이션 =====
const float INIT_ANGLE_A = 90;
const int INIT_ANGLE_B = 93;
const int INIT_ANGLE_C = 87;

const double BASE_HEIGHT = 3.74;
double IK_OFFSET = 0;

const int TARGET_X = 498;
const int TARGET_Y = 515;

// ===== [3] PID 제어 변수 =====
double Kp = 0.0003;
double Kd = 0.023;
double lastErrorX = 0;
double lastErrorY = 0;
float smoothA = INIT_ANGLE_A;
float smoothB = INIT_ANGLE_B;
float smoothC = INIT_ANGLE_C;
const float FILTER_VAL = 0.12;
```

```

// ===== [4] RC카 모터 핀 설정 =====
const int ENA_PIN = 2;
const int IN1_PIN = 6;
const int IN2_PIN = 7;
const int IN3_PIN = 8;
const int IN4_PIN = 9;
const int ENB_PIN = 10;

// □ 스무스 드라이빙 제어 변수
float targetSpeedL = 0; // 왼쪽 모터 목표 속도
float targetSpeedR = 0; // 오른쪽 모터 목표 속도
float currentSpeedL = 0; // 왼쪽 모터 현재 속도
float currentSpeedR = 0; // 오른쪽 모터 현재 속도
const float ACCEL_STEP = 15.0; // 가감속 부드러움 정도 (작을수록 미끄러지듯, 클수록 빠릿하게)

// ===== [5] 자이로 센서 (MPU6050) =====
const int MPU_ADDR = 0x68;
float gyroPitch = 0;
float gyroRoll = 0;
unsigned long gyroLastTime = 0;

// 멀티태스킹 타이머
unsigned long lastLoopTime = 0;

// 함수 선언
void setupMPU();
void updateGyro();
void pidBalance(int rawX, int rawY);
int readTouchX();
int readTouchY();
void updateSmoothMotors();

void setup()
{
    Serial.begin(115200);
    Serial1.begin(9600);

    delay(1000);

```

```

Serial.println("=====");
Serial.println("[System] Booting up...");

setupMPU();

IK_OFFSET = machine.theta(0, BASE_HEIGHT, 0, 0);

servoA.attach(SERVO_PIN_A);
servoB.attach(SERVO_PIN_B);
servoC.attach(SERVO_PIN_C);

servoA.write(INIT_ANGLE_A);
servoB.write(INIT_ANGLE_B);
servoC.write(INIT_ANGLE_C);

pinMode(ENA_PIN, OUTPUT);
pinMode(IN1_PIN, OUTPUT);
pinMode(IN2_PIN, OUTPUT);
pinMode(ENB_PIN, OUTPUT);
pinMode(IN3_PIN, OUTPUT);
pinMode(IN4_PIN, OUTPUT);

Serial.println("=====");
Serial.println("[System] Smooth Drive (Max 180) Ready!");
Serial.println("=====");

lastLoopTime = millis();
gyroLastTime = millis();
}

void loop()
{
    // □ [태스크 1] 블루투스 명령 수신 (목표 최고 속도 180 제한)
    if (Serial1.available())
    {
        char data = Serial1.read();

        // □ 최고 속도를 180, 최저 속도(후진)를 -180으로 변경했습니다.
        if (data == 'F') { targetSpeedL = 190; targetSpeedR = 190; }
        else if (data == 'B') { targetSpeedL = -190; targetSpeedR = -190; }
        else if (data == 'L') { targetSpeedL = -190; targetSpeedR = 190; }
    }
}

```

```

        else if (data == 'R') { targetSpeedL = 190; targetSpeedR = -190; }
        else if (data == 'S') { targetSpeedL = 0; targetSpeedR = 0; }
    }

    // □ [태스크 2] 메인 루프 (20ms 간격) - 여기서 밸런싱과 스무스 주행이 동시에
    이루어집니다.
    if (millis() - lastLoopTime >= 20)
    {
        lastLoopTime = millis();
        updateGyro();

        // □ 현재 속도를 목표 속도로 아주 부드럽게 끌어올리거나 내리는 핵심 함수
        updateSmoothMotors();

        int tx = readTouchX();
        int ty = readTouchY();

        if (tx > 30 && tx < 1000 && ty > 30 && ty < 1000)
        {
            pidBalance(tx, ty);
        }
        else
        {
            smoothA = (smoothA * 0.90) + (INIT_ANGLE_A * 0.10);
            smoothB = (smoothB * 0.90) + (INIT_ANGLE_B * 0.10);
            smoothC = (smoothC * 0.90) + (INIT_ANGLE_C * 0.10);

            servoA.write(smoothA);
            servoB.write(smoothB);
            servoC.write(smoothC);

            lastErrorX = 0;
            lastErrorY = 0;
        }
    }
}

// =====
// □ 멈춤/출발 방지 스무스 모터 업데이트 함수
// =====
void updateSmoothMotors() {

```

```

// 1. 왼쪽 모터 부드러운 가감속
if (currentSpeedL < targetSpeedL) {
    currentSpeedL += ACCEL_STEP;
    if (currentSpeedL > targetSpeedL) currentSpeedL = targetSpeedL;
} else if (currentSpeedL > targetSpeedL) {
    currentSpeedL -= ACCEL_STEP;
    if (currentSpeedL < targetSpeedL) currentSpeedL = targetSpeedL;
}

// 2. 오른쪽 모터 부드러운 가감속
if (currentSpeedR < targetSpeedR) {
    currentSpeedR += ACCEL_STEP;
    if (currentSpeedR > targetSpeedR) currentSpeedR = targetSpeedR;
} else if (currentSpeedR > targetSpeedR) {
    currentSpeedR -= ACCEL_STEP;
    if (currentSpeedR < targetSpeedR) currentSpeedR = targetSpeedR;
}

// 3. 실제 모터 핀에 출력 적용 (왼쪽)
if (currentSpeedL >= 0) {
    digitalWrite(IN1_PIN, HIGH); digitalWrite(IN2_PIN, LOW);
    analogWrite(ENA_PIN, (int)currentSpeedL);
} else {
    digitalWrite(IN1_PIN, LOW); digitalWrite(IN2_PIN, HIGH);
    analogWrite(ENA_PIN, (int)(-currentSpeedL));
}

// 4. 실제 모터 핀에 출력 적용 (오른쪽)
if (currentSpeedR >= 0) {
    digitalWrite(IN3_PIN, HIGH); digitalWrite(IN4_PIN, LOW);
    analogWrite(ENB_PIN, (int)currentSpeedR);
} else {
    digitalWrite(IN3_PIN, LOW); digitalWrite(IN4_PIN, HIGH);
    analogWrite(ENB_PIN, (int)(-currentSpeedR));
}
}

void pidBalance(int rawX, int rawY)
{
    double errorX = rawX - TARGET_X;
    double errorY = rawY - TARGET_Y;

```

```

if (abs(errorX) < 30) errorX = 0;
if (abs(errorY) < 30) errorY = 0;

double derivX = errorX - lastErrorX;
double derivY = errorY - lastErrorY;

double outputX = (Kp * errorX) + (Kd * derivX);
double outputY = (Kp * errorY) + (Kd * derivY);

double compX = outputX - ((gyroRoll + 0.24) * 0.4);
double compY = outputY - ((gyroPitch + 0.03) * 0.4);

compX = constrain(compX, -0.20, 0.20);
compY = constrain(compY, -0.252, 0.28);

double da = machine.theta(0, BASE_HEIGHT, -compX, -compY);
double db = machine.theta(1, BASE_HEIGHT, -compX, -compY);
double dc = machine.theta(2, BASE_HEIGHT, -compX, -compY);

if (!isnan(da) && !isnan(db) && !isnan(dc))
{
    float targetA = INIT_ANGLE_A + (da - IK_OFFSET);
    float targetB = INIT_ANGLE_B + (db - IK_OFFSET);
    float targetC = INIT_ANGLE_C + (dc - IK_OFFSET);

    targetA = constrain(targetA, 10, 105);
    targetB = constrain(targetB, 10, 170);
    targetC = constrain(targetC, 10, 170);

    smoothA = (smoothA * (1.0 - FILTER_VAL)) + (targetA * FILTER_VAL);
    smoothB = (smoothB * (1.0 - FILTER_VAL)) + (targetB * FILTER_VAL);
    smoothC = (smoothC * (1.0 - FILTER_VAL)) + (targetC * FILTER_VAL);

    servoA.write(smoothA);
    servoB.write(smoothB);
    servoC.write(smoothC);
}

lastErrorX = errorX;
lastErrorY = errorY;

```

```

}

// =====
// □ MPU6050 제어 및 터치 제어 함수
// =====

void setupMPU() {
    Wire.begin();
    Wire.beginTransmission(MPU_ADDR);
    Wire.write(0x6B);
    Wire.write(0);
    Wire.endTransmission(true);
}

void updateGyro() {
    Wire.beginTransmission(MPU_ADDR);
    Wire.write(0x3B);
    Wire.endTransmission(false);
    Wire.requestFrom(MPU_ADDR, 14, true);

    int16_t AcX = Wire.read() << 8 | Wire.read();
    int16_t AcY = Wire.read() << 8 | Wire.read();
    int16_t AcZ = Wire.read() << 8 | Wire.read();
    Wire.read(); Wire.read();
    int16_t GyX = Wire.read() << 8 | Wire.read();
    int16_t GyY = Wire.read() << 8 | Wire.read();
    int16_t GyZ = Wire.read() << 8 | Wire.read();

    double dt = (millis() - gyroLastTime) / 1000.0;
    gyroLastTime = millis();

    float accelPitch = atan2(-AcX, sqrt((long)AcY * AcY + (long)AcZ * AcZ));
    float accelRoll = atan2(AcY, AcZ);

    float gyroRateX = (GyX / 131.0) * (PI / 180.0);
    float gyroRateY = (GyY / 131.0) * (PI / 180.0);

    gyroPitch = 0.98 * (gyroPitch + gyroRateY * dt) + 0.02 * accelPitch;
    gyroRoll = 0.98 * (gyroRoll + gyroRateX * dt) + 0.02 * accelRoll;
}

int readTouchX() {

```

```

    pinMode(XP, OUTPUT); digitalWrite(XP, HIGH);
    pinMode(XM, OUTPUT); digitalWrite(XM, LOW);
    pinMode(YP, INPUT_PULLDOWN); pinMode(YM, INPUT);
    delayMicroseconds(50); return analogRead(YP);
}

int readTouchY() {
    pinMode(YP, OUTPUT); digitalWrite(YP, HIGH);
    pinMode(YM, OUTPUT); digitalWrite(YM, LOW);
    pinMode(XP, INPUT_PULLDOWN); pinMode(XM, INPUT);
    delayMicroseconds(50); return analogRead(XP);
}

```

B. Inverse kinematics .h 소스코드

```

#ifndef InverseKinematics_H
#define InverseKinematics_H
#include <Arduino.h>

//constants
#define A 0
#define B 1
#define C 2

class Machine { //machine class
public:
    //class functions
    Machine(double d, double e, double f, double g);
    double theta(int leg, double hz, double nx, double ny); //returns the value of
    theta a, b, or c
};

#endif

```

C. Inverse kinematics .cpp 소스코드

```
#include "InverseKinematics.h"

//Global User Defined Constants
double d; //distance from the center of the base to any of its corners
double e; //distance from the center of the platform to any of its corners
double f; //length of link #1
double g; //length of link #2

//Calculation Variables
double nmag, nz; //magnitude and z component of the normal vector
double x, y, z; //generic variables for the components of legs A, B, and C
double mag; //generic magnitude of the leg vector
double angle; //generic angle for legs A, B, and C

//Library Content
Machine::Machine(double _d, double _e, double _f, double _g) {
    d = _d;
    e = _e;
    f = _f;
    g = _g;
}

double Machine::theta(int leg, double hz, double nx, double ny) {
    //create unit normal vector
    nmag = sqrt(pow(nx, 2) + pow(ny, 2) + 1); //magnitude of the normal vector
    nx /= nmag;
    ny /= nmag;
    nz = 1 / nmag;
    //calculates angle A, B, or C
    switch (leg) {
        case A: //Leg A
            y = d + (e / 2) * (1 - (pow(nx, 2) + 3 * pow(nz, 2) + 3 * nz) / (nz + 1 - pow(nx, 2) + (pow(nx, 4) - 3 * pow(nx, 2) * pow(ny, 2)) / ((nz + 1) * (nz + 1 - pow(nx, 2))))));
            z = hz + e * ny;
            mag = sqrt(pow(y, 2) + pow(z, 2));
            angle = acos(y / mag) + acos((pow(mag, 2) + pow(f, 2) - pow(g, 2)) / (2 * mag * f));
            break;
        case B: //Leg B
            x = (sqrt(3) / 2) * (e * (1 - (pow(nx, 2) + sqrt(3) * nx * ny) / (nz + 1)) - d);
```

```

y = x / sqrt(3);
    z = hz - (e / 2) * (sqrt(3) * nx + ny);
    mag = sqrt(pow(x, 2) + pow(y, 2) + pow(z, 2));
    angle = acos((sqrt(3) * x + y) / (-2 * mag)) + acos((pow(mag, 2) +
pow(f, 2) - pow(g, 2)) / (2 * mag * f));
    break;
case C: //Leg C
    x = (sqrt(3) / 2) * (d - e * (1 - (pow(nx, 2) - sqrt(3) * nx * ny) / (nz
+ 1)));
    y = -x / sqrt(3);
    z = hz + (e / 2) * (sqrt(3) * nx - ny);
    mag = sqrt(pow(x, 2) + pow(y, 2) + pow(z, 2));
    angle = acos((sqrt(3) * x - y) / (2 * mag)) + acos((pow(mag, 2) + pow(f,
2) - pow(g, 2)) / (2 * mag * f));
    break;
}
return (angle * (180 / PI)); //converts angle to degrees and returns the value
}

```